

UNIVERSIDAD DE GUADALAJARA

Centro Universitario de Ciencias Exactas e Ingenierías



Actividad 04 - Sumador Nibble y Sumador 8 bits

Por:

ESTEFANIA NAVARRO MENDOZA

Docente:

JORGE ERNESTO LOPEZ ARCE DELGADO

1. Introducción

La Unidad Aritmético Lógica (ALU) es uno de los componentes más críticos de un procesador, ya que se encarga de realizar todas las operaciones lógicas y matemáticas fundamentales. En el diseño de sistemas digitales, la operación de suma se considera la base de casi toda la aritmética, pues a través de ella se pueden derivar restas, multiplicaciones y divisiones.

Para la representación de números negativos y la realización de operaciones complejas, se utilizan diversos sistemas:

- **Complemento a 1:** Consiste en invertir todos los bits de un número binario (los 0 pasan a 1 y viceversa).
- **Complemento a 2:** Es el estándar actual en la computación; se obtiene sumando un 1 al bit menos significativo del complemento a 1. Este sistema facilita las operaciones de resta utilizando el mismo hardware del sumador.
- **Punto Flotante:** Es la representación necesaria para manejar números reales (con decimales) y rangos muy amplios de valores, siguiendo generalmente el estándar IEEE 754.

2. Objetivos

Objetivo General: Diseñar e implementar módulos de aritmética computacional en lenguaje Verilog para comprender el funcionamiento de una Unidad Aritmético Lógica (ALU) y la importancia de la operación de suma como base del procesador.

Objetivos Particulares:

- Implementar un sumador de 4 bits y uno de 8 bits mediante el uso de instancias de módulos previamente desarrollados (diseño jerárquico).
- Desarrollar versiones comportamentales de los sumadores para comparar diferentes métodos de descripción de hardware en Verilog.
- Validar el diseño mediante bancos de prueba (testbenches) realizando 5 operaciones distintas por sumador y visualizando los resultados en formato decimal.
- Diferenciar las características y aplicaciones de los sumadores implementados en arquitecturas de CPU y GPU.

3. Desarrollo

Para el cumplimiento de esta actividad, se siguió una metodología de diseño modular (bottom-up), comenzando desde las unidades más simples hasta completar los sumadores de mayor dificultad.

1. Implementación de Módulos Base

Se inició con la creación del Medio Sumador (HA.v), el cual es el único módulo que no contiene instancias adicionales. Este utiliza una compuerta XOR para la suma y una AND para el acarreo. Posteriormente, se desarrolló el Sumador Completo (FA.v) instanciando dos módulos HA y una compuerta OR para gestionar el acarreo de entrada, permitiendo así la conexión en cascada.

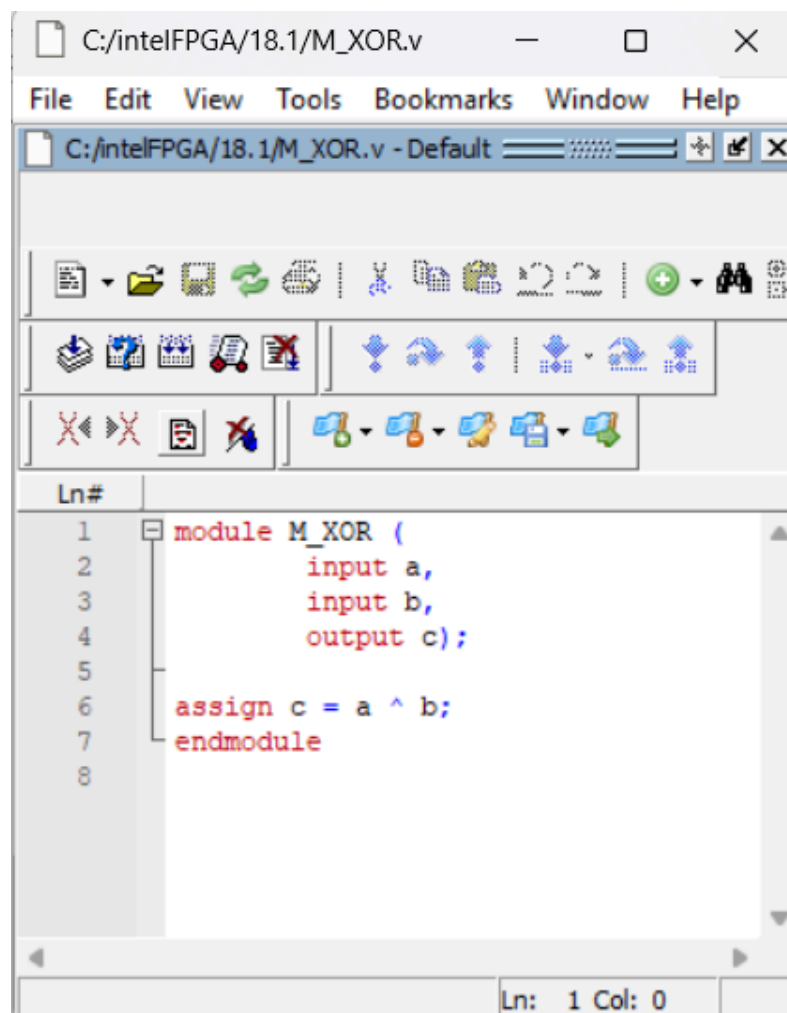


Imagen 1. Implementación del módulo de compuerta lógica XOR

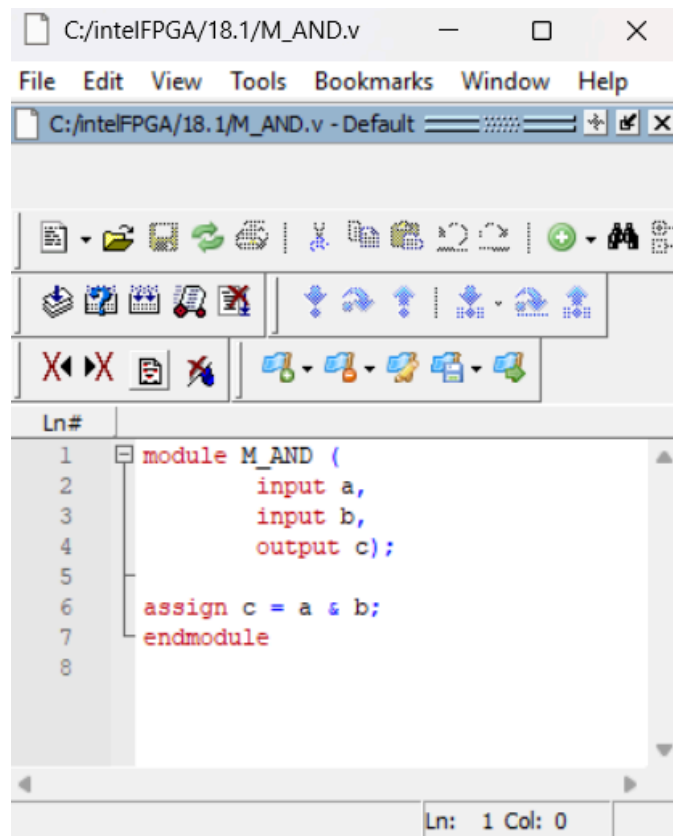


Imagen 2. Implementación del módulo de compuerta lógica AND

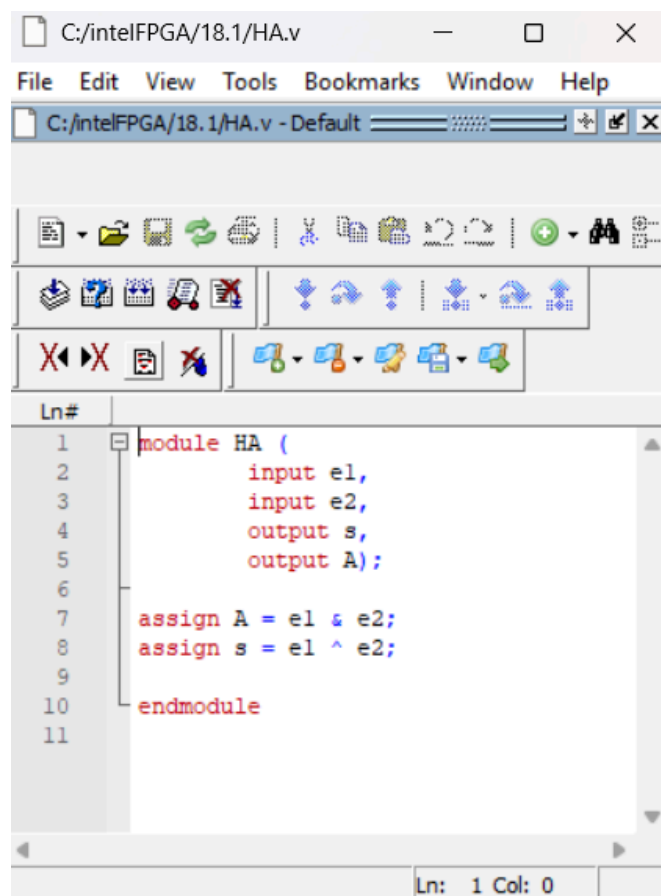


Imagen 3. Código fuente del Medio Sumador (HA . v) utilizando asignaciones continuas para suma y acarreo.

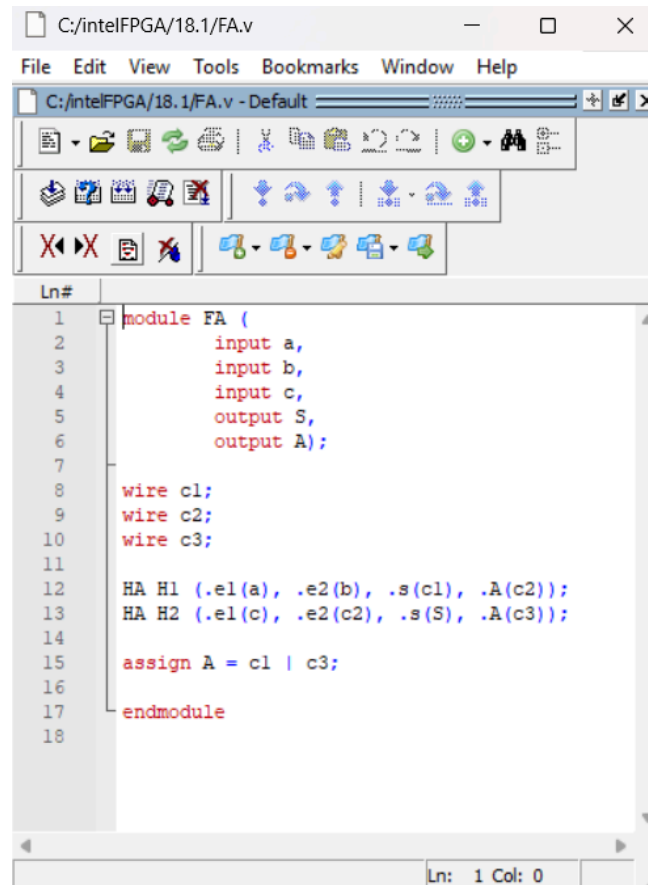


Imagen 4. Diseño modular del Sumador Completo (FA . v) mediante la instancia de dos módulos HA.

2. Sumadores Instanciados (Estructurales)

Utilizando los módulos FA.v, se construyeron los sumadores de 4 y 8 bits:

- Sum_instancia4b.v: Se conectaron 4 sumadores completos en serie, donde el acarreo de salida de cada bit se conecta al acarreo de entrada del bit siguiente.
- Sum_instancia8b.v: Siguiendo la misma lógica, se extendió el diseño para un ancho de palabra de 8 bits, manejando buses de entrada y un resultado de 9 bits (8 de suma y 1 de acarreo final).

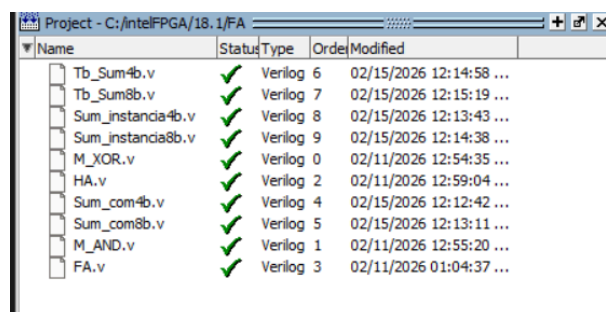


Imagen 5. Estado del proyecto en ModelSim mostrando la compilación exitosa de todos los módulos del diseño.

```

C:/intelFPGA/18.1/Sum_instancia4b.v
File Edit View Tools Bookmarks Window Help
C:/intelFPGA/18.1/Sum_instancia4b.v - Default
Ln#
1 module Sum_instancia4b (
2     input [3:0] A,
3     input [3:0] B,
4     input Ci4,
5     output [3:0] S,
6     output Co4
7 );
8     wire c1, c2, c3;
9
10    FA F0 (.a(A[0]), .b(B[0]), .c(Ci4), .S(S[0]), .A(c1));
11    FA F1 (.a(A[1]), .b(B[1]), .c(c1), .S(S[1]), .A(c2));
12    FA F2 (.a(A[2]), .b(B[2]), .c(c2), .S(S[2]), .A(c3));
13    FA F3 (.a(A[3]), .b(B[3]), .c(c3), .S(S[3]), .A(Co4));
14 endmodule
15
Ln: 1 Col: 0

```

Imagen 6. Diseño jerárquico del Sumador de 4 bits.

```

C:/intelFPGA/18.1/Sum_instancia8b.v
File Edit View Tools Bookmarks Window Help
C:/intelFPGA/18.1/Sum_instancia8b.v - Default
Ln#
1 module Sum_instancia8b (
2     input [7:0] e1,
3     input [7:0] e2,
4     input Ci8,
5     output [7:0] S8,
6     output Co8
7 );
8     wire c_inter;
9
10    Sum_instancia4b ins41 (.A(e1[3:0]), .B(e2[3:0]), .Ci4(Ci8), .S(S8[3:0]), .Co4(c_inter));
11    Sum_instancia4b ins42 (.A(e1[7:4]), .B(e2[7:4]), .Ci4(c_inter), .S(S8[7:4]), .Co4(Co8));
12 endmodule
13
Ln: 1 Col: 0

```

Imagen 7. Diseño jerárquico del Sumador de 8 bits conectando dos módulos de 4 bits en cascada.

3. Sumadores Comportamentales

Se crearon los módulos Sum_com4b.v y Sum_com8b.v. A diferencia de los anteriores, en estos se utilizó el operador aritmético directo de Verilog (+), lo que permite al software de síntesis optimizar la lógica automáticamente.

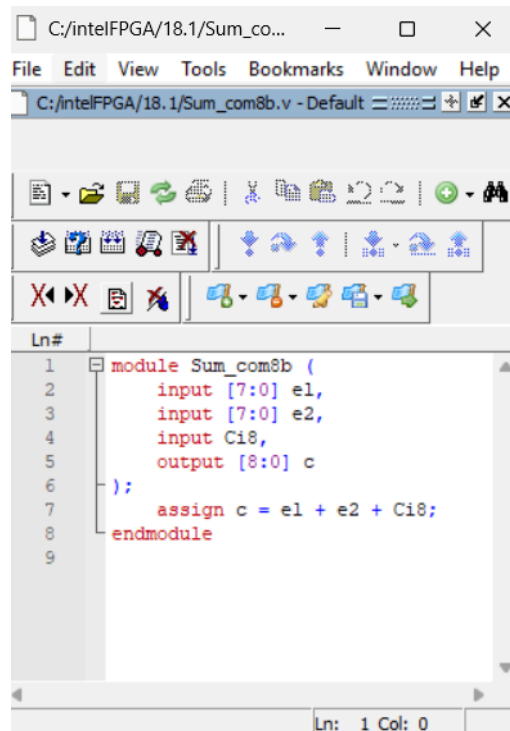


Imagen 8. Implementación comportamental del Sumador de 8 bits para validación de lógica simplificada.

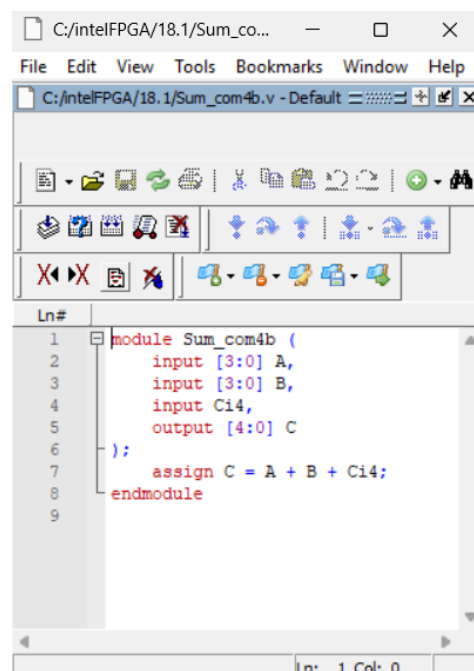


Imagen 9. Código comportamental del Sumador de 4 bits utilizando el operador aritmético directo.

4. Simulación y Validación

La validación se realizó en el software ModelSim. Se crearon dos archivos de prueba: Tb_Sum4b.v y Tb_Sum8b.v. En cada uno se programaron 5 casos de prueba con diferentes valores decimales para asegurar que el acarreo se propague correctamente y que el resultado final sea exacto.

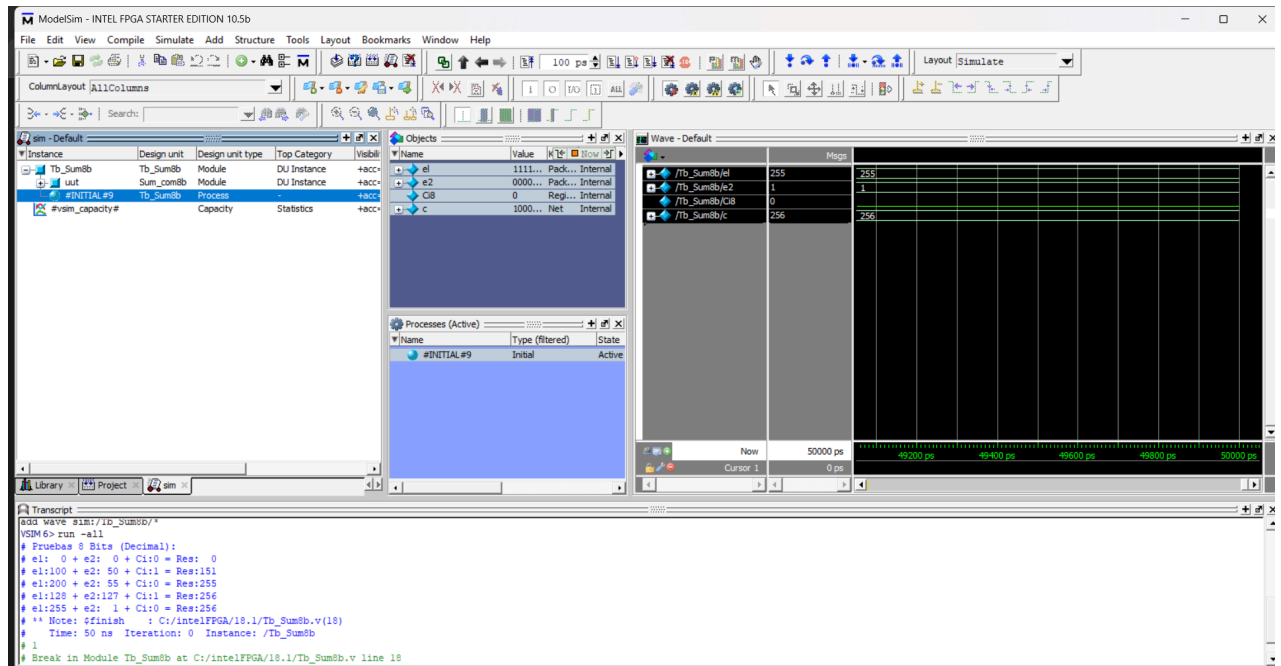


Imagen 10. Código del Banco de Pruebas (Tb_Sum8b.v) con la definición de los 5 casos de prueba y directivas de monitoreo.

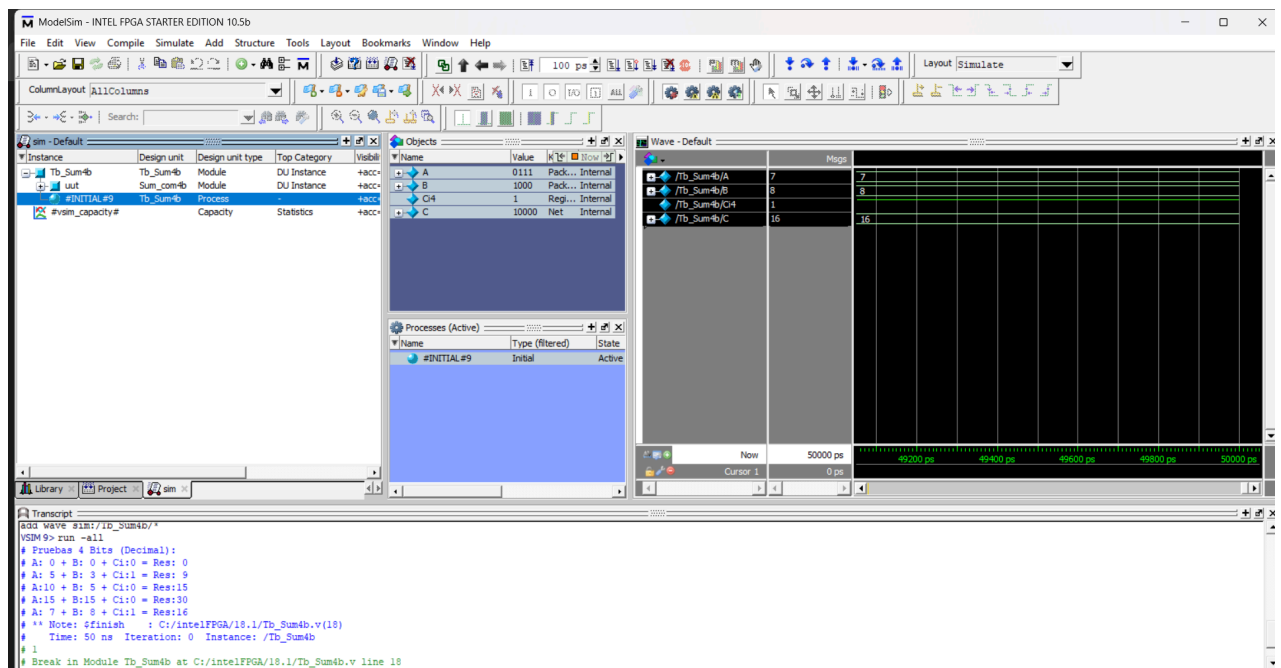


Imagen 11. Código del Banco de Pruebas (Tb_Sum4b.v) con la definición de los 5 casos de prueba y directivas de monitoreo.

4. Conclusión

Al finalizar esta práctica, logré comprender cómo la suma binaria es el pilar fundamental de la Unidad Aritmético Lógica. La implementación modular me permitió ver físicamente cómo se propaga el acarreo a través de los diferentes bits, lo cual es esencial para entender las limitaciones de velocidad en los sumadores de tipo "Ripple Carry" usados en algunas CPUs, en contraste con los diseños más paralelos que podrían encontrarse en una GPU.

Un aspecto clave fue el manejo de buses y vectores en Verilog, ya que el paso de parámetros entre módulos requiere una organización estricta de los bits. El uso de testbenches con visualización decimal facilitó enormemente la detección de errores en la lógica de acarreo. Considero que esta actividad fue fundamental para pasar de la teoría de compuertas a un diseño funcional de arquitectura computacional.

5. Referencias

- Mano, M. M., & Ciletti, M. D. (2017). *Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog*. Pearson.
- Guía de la Actividad 04: Aritmética computacional. Seminario De Solución De Problemas De Arquitectura De Computadoras, Universidad de Guadalajara.